

NAT

Quantitative Research Platform

Mathematical Foundations, Algorithms &
Autonomous Agent Architecture

Self-Study Reference — Revision 1

Prepared by Yigit Onat

May 23, 2026

Contents

1	Information Theory	4
1.1	Shannon Entropy	4
1.2	Permutation Entropy	4
1.3	Tick Direction Entropy	4
1.4	Mutual Information (KSG Estimator)	5
1.5	Conditional Mutual Information	5
1.6	Interaction Information	6
1.7	Transfer Entropy	6
1.8	Information Cost Function	6
1.9	Kullback-Leibler Divergence	6
2	Stochastic Processes & State-Space Filtering	8
2.1	Ornstein-Uhlenbeck Process	8
2.2	Kalman Filter	8
2.3	Regime-Switching Kalman Filter (Hamilton Filter)	9
2.4	Hawkes Self-Exciting Process	9
2.5	Sequential Probability Ratio Test (SPRT)	10
3	Volatility Estimation & Jump Detection	11
3.1	Realized Volatility	11
3.2	Parkinson Volatility (1980)	11
3.3	Garman-Klass Volatility (1980)	11
3.4	Bipower Variation (BNS 2004)	11
3.5	Lee-Mykland Jump Detection (2008)	12
4	Market Microstructure Theory	13
4.1	Kyle's Lambda (1985)	13
4.2	Amihud Illiquidity (2002)	13
4.3	VPIN (Volume-Synchronized Probability of Informed Trading)	13
4.4	Bouchaud Propagator Model (2004)	13
4.5	Huang-Stoll Spread Decomposition (1997)	14
4.6	Hasbrouck's Lambda (2009)	14
5	Concentration & Inequality Metrics	15
5.1	Herfindahl-Hirschman Index (HHI)	15
5.2	Gini Coefficient	15
5.3	Theil Index	15
6	Statistical Hypothesis Testing	16
6.1	Correlation Testing	16
6.2	Walk-Forward Validation	16
6.3	Multiple Testing Correction	16
7	Online Learning Algorithms	17
7.1	Online Ridge Regression	17
8	Regime Detection & Classification	18
8.1	Gaussian Mixture Model (GMM)	18
8.2	Absorption Ratio	18
8.3	Regime Composite Score	18

9	Trend & Time Series Analysis	19
9.1	Hurst Exponent	19
9.2	Momentum (Linear Regression Slope)	19
9.3	EMA Crossover	19
10	Complete Algorithm Inventory	20
10.1	Implemented Streaming Algorithms (26)	20
10.2	Algorithms Not Yet Implemented (High-Value Candidates)	20
11	Autonomous Feature Engineering Agent	22
11.1	Current Agent Architecture	22
11.2	Proposed Extension: Feature Factory Agent	22
11.2.1	Generator Operations (Feature Algebra)	22
11.2.2	Evaluation Metrics	22
11.2.3	Selection Protocol (Extended 5-Gate)	23
11.2.4	Evolution Step (Missing Piece)	23
11.2.5	Architecture	23
11.2.6	Progression: Semi-Autonomous to Autonomous	23
12	Signal Processing: FIR, IIR & Advanced Filtering	24
12.1	Finite Impulse Response (FIR) Filters	24
12.2	Infinite Impulse Response (IIR) Filters	24
12.3	Butterworth Filter	25
12.4	Relationship to Existing NAT Filters	25
13	NEAR Protocol: Technical Analysis	26
13.1	Architecture	26
13.2	Strengths vs. Other L1s	26
13.3	Relevance for Quantitative Trading	26
13.4	Potential Contribution Areas	26
14	Hermes Agents & Evolutionary Self-Learning	27
14.1	Hermes Agent (Nous Research)	27
14.2	Comparison with NAT Agent	27
14.3	Related Evolutionary Approaches	27
14.4	The Generate-Test-Prune-Evolve Loop	27
15	Evolutionary & AutoML Frameworks for Strategy Discovery	28
15.1	Foundations: Evolutionary Computation	28
15.1.1	Core Loop	28
15.1.2	Example: Evolving a Trading Signal Weight Vector	28
15.1.3	Selection Methods	29
15.1.4	Quality-Diversity (MAP-Elites)	29
15.2	Foundations: Genetic Programming (GP)	29
15.2.1	Tree Representation	29
15.2.2	GP Crossover	30
15.2.3	GP Mutation	30
15.2.4	Bloat Control	31
15.2.5	Vectorial GP: The Key Innovation	31
15.3	Foundations: AutoML and Self-Improving Systems	31
15.3.1	The Closed-Loop Self-Improvement Pattern	31
15.3.2	Concrete Example: Feature Factory Learning	32
15.3.3	Early-Stop Predictor	32

15.3.4	RL Reward Shaping for Feature Generation	33
15.4	QuantEvolve: Multi-Agent Evolutionary Strategy Discovery	33
15.4.1	Architecture	33
15.4.2	Feature Map Archive (MAP-Elites)	33
15.4.3	Evolution Operators	34
15.4.4	Fitness Function	34
15.4.5	Key Results	34
15.4.6	Relation to AlphaEvolve	34
15.4.7	Limitations	34
15.4.8	Relevance to NAT	34
15.5	Vectorial Genetic Programming for Trading Rules	35
15.5.1	Vectorial GP vs. Standard GP	35
15.5.2	Terminal and Function Sets	35
15.5.3	Three Variants	35
15.5.4	Fitness Function	35
15.5.5	Overfitting Mitigation	35
15.5.6	GP Parameters	36
15.5.7	Key Results	36
15.5.8	Relevance to NAT	36
15.6	NNGPT: LLM as Self-Improving AutoML Engine	36
15.6.1	Core Concept	36
15.6.2	Closed-Loop Architecture	36
15.6.3	Five Integrated Pipelines	37
15.6.4	Self-Improvement Mechanism	37
15.6.5	Key Results	37
15.6.6	Limitations	38
15.6.7	Relevance to NAT	38
15.7	Synthesis: What NAT Should Adopt	39

16 Recommended Study Roadmap 40

1 Information Theory

Information theory provides the mathematical framework for quantifying uncertainty, dependence, and information flow in the feature space. NAT uses it both for feature computation (entropy features) and for feature selection (mutual information, transfer entropy).

1.1 Shannon Entropy

The fundamental measure of uncertainty in a discrete random variable X with probability mass function $p(x)$:

$$H(X) = - \sum_{x \in \mathcal{X}} p(x) \log p(x) \quad (1)$$

Properties:

- $H(X) \geq 0$, with equality iff X is deterministic
- $H(X) \leq \log |\mathcal{X}|$, with equality iff X is uniform
- Concave in p

Implementation: `rust/ing/src/features/entropy.rs`. NAT computes distribution entropy by binning continuous values (spreads, volumes, book shape) into N bins, computing empirical $\hat{p}_i = n_i/n$, and normalizing by $\log N$ to yield $H_{\text{norm}} \in [0, 1]$.

Features: `spread_dispersion`, `volume_dispersion`, `book_shape`, `trade_size_dispersion`.

1.2 Permutation Entropy

Introduced by Bandt & Pompe (2002), permutation entropy captures the complexity of a time series through its ordinal pattern structure. Given an embedding dimension m and delay τ , each subsequence $(x_t, x_{t+\tau}, \dots, x_{t+(m-1)\tau})$ maps to one of $m!$ possible ordinal patterns π .

$$H_{\text{PE}}(m, \tau) = - \sum_{\pi \in S_m} p(\pi) \log p(\pi) \quad (2)$$

where S_m is the symmetric group of order m and $p(\pi)$ is the relative frequency of pattern π .

Normalized: $h_{\text{PE}} = H_{\text{PE}} / \log(m!)$, giving $h_{\text{PE}} \in [0, 1]$.

Implementation: NAT uses $m = 3$ (yielding $3! = 6$ possible patterns) with windows of 8, 16, and 32 ticks. Features: `permutation_returns_8/16/32`, `permutation_imbalance_16`.

Interpretation:

- $h_{\text{PE}} \approx 1$: all ordinal patterns equally likely $\Rightarrow$ random, unpredictable
- $h_{\text{PE}} \ll 1$: few patterns dominate $\Rightarrow$ structured, predictable

Reference: Bandt, C. & Pompe, B. (2002). Permutation entropy: a natural complexity measure for time series. *Physical Review Letters*, 88(17).

1.3 Tick Direction Entropy

A domain-specific entropy measuring the predictability of trade direction. Each trade is classified as {up, down, neutral} using the tick rule (comparison with previous trade price), yielding a trinomial distribution.

$$H_{\text{tick}}(w) = - \sum_{d \in \{u, d, n\}} p_d(w) \log p_d(w) \quad (3)$$

where w is the time window and $p_d(w)$ is the fraction of ticks in direction d within window w .

Range: $[0, \log 3] \approx [0, 1.099]$.

Implementation: Computed at 7 timescales: 1s, 5s, 10s, 15s, 30s, 1m, 15m. Volume-weighted variants weight each tick by trade size.

Features: `tick_entropy_1s` through `tick_entropy_15m`, `volume_tick_entropy_1s` through `volume_tick_entropy_15m` (14 features total).

1.4 Mutual Information (KSG Estimator)

Mutual information quantifies the total (linear + nonlinear) statistical dependence between two random variables:

$$I(X; Y) = H(X) + H(Y) - H(X, Y) \quad (4)$$

Equivalently, $I(X; Y) = D_{\text{KL}}(p_{XY} \| p_X p_Y)$, the Kullback-Leibler divergence between the joint and product of marginals.

KSG Algorithm 1 (Kraskov, Stögbauer, Grassberger, 2004): A k -nearest-neighbor estimator that avoids explicit density estimation. For each sample (x_i, y_i) :

1. Find the k -th nearest neighbor in the joint space using Chebyshev (max-norm) distance:
 $\epsilon(i) = \|z_i - z_i^{(k)}\|_\infty$
2. Count $n_x(i)$: number of points with $\|x_j - x_i\|_\infty < \epsilon(i)$
3. Count $n_y(i)$: number of points with $\|y_j - y_i\|_\infty < \epsilon(i)$

$$\hat{I}_{\text{KSG}}(X; Y) = \psi(k) - \frac{1}{N} \sum_{i=1}^N [\psi(n_x(i) + 1) + \psi(n_y(i) + 1)] + \psi(N) \quad (5)$$

where $\psi(\cdot)$ is the digamma function.

Properties:

- Bias-corrected: $\hat{I} \geq 0$ by construction
- Nonparametric: no distributional assumptions
- Consistent: converges as $N \rightarrow \infty$
- Sensitive to k : NAT uses $k = 5$ (default)

Implementation: `scripts/it_engine/estimators.py`. Used for alpha discovery and feature selection.

Reference: Kraskov, A., Stögbauer, H. & Grassberger, P. (2004). Estimating mutual information. *Physical Review E*, 69(6).

1.5 Conditional Mutual Information

CMI measures the dependence between X and Y given knowledge of Z :

$$I(X; Y|Z) = H(X, Z) + H(Y, Z) - H(X, Y, Z) - H(Z) \quad (6)$$

Each entropy term is estimated via the KSG k -NN approach in the appropriate joint space.

Usage in NAT: Measures *marginal* information content — does feature X tell us something about returns Y that the existing feature set Z does not already capture?

Implementation: `scripts/it_engine/estimators.py`, used by the greedy feature selector.

1.6 Interaction Information

A three-variable generalization that distinguishes synergy from redundancy:

$$II(X; Y; Z) = I(X; Y|Z) - I(X; Y) \quad (7)$$

Interpretation:

- $II > 0$: **Synergy** — X and Y together tell more about Z than separately
- $II < 0$: **Redundancy** — X and Y share overlapping information about Z
- $II = 0$: Independence of the interaction

Implementation: `scripts/it_engine/estimators.py`. Used to detect feature combinations that are more powerful together than individually.

1.7 Transfer Entropy

Transfer entropy (Schreiber, 2000) measures directed information flow from a source process X to a target process Y :

$$TE_{X \rightarrow Y}(p, q) = I(Y_{t+1}; X_t^{(q)} | Y_t^{(p)}) \quad (8)$$

where $X_t^{(q)} = (X_t, X_{t-1}, \dots, X_{t-q+1})$ and $Y_t^{(p)} = (Y_t, Y_{t-1}, \dots, Y_{t-p+1})$.

NAT implementation: Linear (autoregressive) approximation:

$$TE_{\text{linear}} = \frac{1}{2} \log \frac{\text{Var}(\varepsilon_{\text{reduced}})}{\text{Var}(\varepsilon_{\text{full}})} \quad (9)$$

where $\varepsilon_{\text{reduced}}$ are residuals from $Y_{t+1} = f(Y_t^{(p)})$ and $\varepsilon_{\text{full}}$ from $Y_{t+1} = g(Y_t^{(p)}, X_t^{(q)})$.

Usage: Detect causal information flow from features to returns. A feature with high $TE_{X \rightarrow \text{returns}}$ carries predictive, not just correlational, information.

Reference: Schreiber, T. (2000). Measuring information transfer. *Physical Review Letters*, 85(2).

1.8 Information Cost Function

A trading-specific threshold: a feature is only viable if its information content exceeds the cost of acting on it.

$$I_{\min}(\text{fee}, \sigma_r) = \frac{1}{2} \log \left(1 + \frac{\text{fee}^2}{\sigma_r^2} \right) \quad (10)$$

where fee is the round-trip transaction cost (in return units) and σ_r is the return volatility.

Implementation: `scripts/it_engine/estimators.py`. Config in `config/it_engine.toml` with exchange-specific fee schedules (e.g., Binance VIP tiers).

Gate rule: Feature passes cost gate iff $I(X; \text{returns}) > I_{\min}$.

1.9 Kullback-Leibler Divergence

The directed divergence (information gain) from distribution Q to P :

$$D_{\text{KL}}(P \| Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)} \quad (11)$$

Properties:

- $D_{\text{KL}} \geq 0$ (Gibbs' inequality), with equality iff $P = Q$
- Not symmetric: $D_{\text{KL}}(P\|Q) \neq D_{\text{KL}}(Q\|P)$ in general
- Connection to MI: $I(X; Y) = D_{\text{KL}}(p_{XY} \| p_X p_Y)$

Used implicitly throughout the information-theoretic framework.

2 Stochastic Processes & State-Space Filtering

2.1 Ornstein-Uhlenbeck Process

The OU process is the continuous-time analog of a discrete AR(1) — a mean-reverting diffusion:

$$dx_t = -\theta(x_t - \mu) dt + \sigma dW_t \quad (12)$$

Parameters:

- $\theta > 0$: mean-reversion speed (half-life = $\ln 2/\theta$)
- μ : long-run mean
- σ : volatility of innovations

Discrete-time transition (exact, for time step Δt):

$$x_{t+1} = \mu + e^{-\theta\Delta t}(x_t - \mu) + \eta_t \quad (13)$$

$$\eta_t \sim \mathcal{N}\left(0, \frac{\sigma^2}{2\theta}(1 - e^{-2\theta\Delta t})\right) \quad (14)$$

Usage in NAT: Models order book imbalance as mean-reverting around zero. Default $\theta \approx 0.1$ (half-life 5–7 seconds at 100ms emission).

2.2 Kalman Filter

The optimal linear state estimator for the linear Gaussian state-space model:

State equation:

$$x_{t+1} = Fx_t + w_t, \quad w_t \sim \mathcal{N}(0, Q) \quad (15)$$

Observation equation:

$$z_t = Hx_t + v_t, \quad v_t \sim \mathcal{N}(0, R) \quad (16)$$

Prediction step:

$$\hat{x}_{t|t-1} = F\hat{x}_{t-1|t-1} \quad (17)$$

$$P_{t|t-1} = FP_{t-1|t-1}F^\top + Q \quad (18)$$

Update step:

$$\nu_t = z_t - H\hat{x}_{t|t-1} \quad (\text{innovation}) \quad (19)$$

$$S_t = HP_{t|t-1}H^\top + R \quad (\text{innovation covariance}) \quad (20)$$

$$K_t = P_{t|t-1}H^\top S_t^{-1} \quad (\text{Kalman gain}) \quad (21)$$

$$\hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t\nu_t \quad (22)$$

$$P_{t|t} = (I - K_tH)P_{t|t-1} \quad (23)$$

OU-specific parameterization (NAT):

$$F = e^{-\theta\Delta t}, \quad H = 1 \quad (24)$$

$$Q = \frac{\sigma^2}{2\theta}(1 - F^2), \quad R = \sigma_{\text{obs}}^2 \quad (25)$$

Implementation: `scripts/algorithms/kalman_imbalance.py`, `scripts/kalman/ou_filter.py`.

Output features:

Feature	Definition
alg_kalman_filtered_imb	$\hat{x}_{t t}$ — filtered slow imbalance
alg_kalman_uncertainty	$P_{t t}$ — posterior variance
alg_kalman_innovation	$\nu_t = z_t - \hat{x}_{t t-1}$
alg_kalman_signal_strength	$\hat{x}_{t t}/\sqrt{P_{t t}}$ — signal-to-noise

2.3 Regime-Switching Kalman Filter (Hamilton Filter)

Two parallel Kalman filters, each modeling a different market regime:

- **Regime 0 (fast):** $\theta_0 = 0.5$ — rapid mean-reversion, range-bound market
- **Regime 1 (slow):** $\theta_1 = 0.05$ — slow reversion, trending market

Transition matrix:

$$\Pi = \begin{pmatrix} 1-\rho & \rho \\ \rho & 1-\rho \end{pmatrix}, \quad \rho = 0.01 \text{ per tick} \quad (26)$$

Bayesian posterior update (Hamilton, 1989):

1. Predict regime probabilities: $\tilde{\pi}_t = \Pi^\top \pi_{t-1}$
2. Compute likelihoods: $f_k(z_t) = \mathcal{N}(z_t; H\hat{x}_{t|t-1}^{(k)}, S_t^{(k)})$ for $k \in \{0, 1\}$
3. Update: $\pi_t^{(k)} = \frac{f_k(z_t) \cdot \tilde{\pi}_t^{(k)}}{\sum_j f_j(z_t) \cdot \tilde{\pi}_t^{(j)}}$

Output: Probability-weighted state estimate: $\hat{x}_t = \pi_t^{(0)} \hat{x}_t^{(0)} + \pi_t^{(1)} \hat{x}_t^{(1)}$.

Implementation: `scripts/algorithms/switching_ou.py`.

Reference: Hamilton, J.D. (1989). A new approach to the economic analysis of nonstationary time series. *Econometrica*, 57(2).

2.4 Hawkes Self-Exciting Process

A point process where past events increase the probability of future events (clustering):

$$\lambda(t) = \mu + \sum_{t_i < t} \alpha e^{-\beta(t-t_i)} \quad (27)$$

Parameters:

- $\mu > 0$: baseline intensity (events per second)
- $\alpha > 0$: excitation magnitude per event
- $\beta > 0$: excitation decay rate
- α/β : branching ratio (if > 1 , process is explosive)

Recursive computation (efficient online update):

$$A(t) = e^{-\beta(t-t_{\text{prev}})} \cdot A(t_{\text{prev}}) + N(t) \quad (28)$$

$$\lambda(t) = \mu + \alpha \cdot A(t) \quad (29)$$

where $N(t)$ is the number of events at time t and $A(t)$ is the accumulated excitation state.

Excitement fraction:

$$\varphi(t) = \frac{\alpha \cdot A(t)}{\lambda(t)} = \frac{\lambda(t) - \mu}{\lambda(t)} \in [0, 1] \quad (30)$$

Measures what fraction of current intensity is due to self-excitation vs. baseline.

NAT parameterization: $\alpha = 0.5/\text{sec}$, $\beta = 1.0/\text{sec}$ (calibrated for HFT-scale trade arrival).
Branching ratio $= \alpha/\beta = 0.5$ (subcritical, stable).

Bid/Ask decomposition: Separate excitation states A^{bid} , A^{ask} using imbalance as a pressure proxy:

$$\text{HI}(t) = \frac{\lambda^{\text{ask}}(t) - \lambda^{\text{bid}}(t)}{\lambda^{\text{ask}}(t) + \lambda^{\text{bid}}(t) + \varepsilon} \quad (31)$$

Implementation: `rust/ing/src/features/hawkes.rs` (feature), `scripts/algorithms/hawkes_intensity.py` (algorithm with bid/ask decomposition).

Reference: Bacry, E., Mastromatteo, I. & Muzy, J.-F. (2015). Hawkes processes in finance. *Market Microstructure and Liquidity*, 1(1).

2.5 Sequential Probability Ratio Test (SPRT)

Wald's (1947) optimal sequential hypothesis test. Given observations $\nu_1, \nu_2, \dots$ (Kalman innovations in NAT):

Hypotheses:

$$H_0 : \nu_t \sim \mathcal{N}(0, \hat{\sigma}^2) \quad (\text{no drift}) \quad (32)$$

$$H_1 : \nu_t \sim \mathcal{N}(\mu, \hat{\sigma}^2) \quad (\text{drift } \mu \text{ detected}) \quad (33)$$

Log-likelihood ratio increment:

$$\Lambda_t = \frac{\mu}{\hat{\sigma}^2} \nu_t - \frac{\mu^2}{2\hat{\sigma}^2} \quad (34)$$

Cumulative statistic:

$$S_n = \sum_{t=1}^n \Lambda_t \quad (35)$$

Decision boundaries:

$$A = \log \frac{1 - \beta}{\alpha} \quad (\text{accept } H_1 \text{ if } S_n \geq A) \quad (36)$$

$$B = \log \frac{\beta}{1 - \alpha} \quad (\text{accept } H_0 \text{ if } S_n \leq B) \quad (37)$$

NAT defaults: $\alpha = 0.05$ (Type I), $\beta = 0.20$ (Type II), $\mu = 0.001$. This gives $A \approx 2.77$, $B \approx -1.55$.

Integration with Kalman: The Kalman filter pre-whitens the imbalance signal to produce i.i.d. innovations ν_t , which the SPRT then tests for drift. The innovation variance is tracked with an EMA: $\hat{\sigma}^2(t) = \alpha_{\text{ema}} \nu_t^2 + (1 - \alpha_{\text{ema}}) \hat{\sigma}^2(t-1)$.

Implementation: `scripts/algorithms/optimal_entry.py`.

Reference: Wald, A. (1947). *Sequential Analysis*. John Wiley & Sons.

3 Volatility Estimation & Jump Detection

3.1 Realized Volatility

The simplest nonparametric volatility estimator from tick-level returns $r_i = \log(p_i/p_{i-1})$:

$$RV_N = \sum_{i=1}^N r_i^2 \quad (38)$$

NAT reports the annualized standard deviation: $\hat{\sigma} = \sqrt{RV_N/N}$.

Windows: 60 ticks (≈ 1 min), 300 ticks (≈ 5 min). Features: `vol_returns_1m`, `vol_returns_5m`.

3.2 Parkinson Volatility (1980)

Uses the high-low range within a window, which is more efficient than close-to-close:

$$\hat{\sigma}_P = \frac{\log(H/L)}{\sqrt{4 \ln 2}} \quad (39)$$

Efficiency: $\approx 5.2\times$ more efficient than close-to-close (uses more information from the price path). Feature: `vol_parkinson_5m`.

Reference: Parkinson, M. (1980). The extreme value method for estimating the variance of the rate of return. *The Journal of Business*, 53(1).

3.3 Garman-Klass Volatility (1980)

Uses OHLC data for maximum efficiency:

$$\hat{\sigma}_{GK}^2 = 0.5 \left(\log \frac{H}{L} \right)^2 - (2 \ln 2 - 1) \left(\log \frac{C}{O} \right)^2 \quad (40)$$

Efficiency: $\approx 7.4\times$ close-to-close, $\approx 1.2\times$ Parkinson. Feature: `vol_garman_klass_5m`.

Reference: Garman, M.B. & Klass, M.J. (1980). On the estimation of security price volatilities from historical data. *The Journal of Business*, 53(1).

3.4 Bipower Variation (BNS 2004)

Separates continuous volatility from jumps:

$$BV_N = \frac{\pi}{2} \cdot \frac{N}{N-1} \sum_{i=2}^N |r_i| \cdot |r_{i-1}| \quad (41)$$

Key property: Under mild conditions, $BV_N \xrightarrow{p} \int_0^T \sigma_t^2 dt$ (integrated variance), regardless of jumps. The realized variance RV converges to integrated variance + jump variation.

Jump decomposition:

$$JV = \max(RV - BV, 0) \quad (42)$$

$$JR = \frac{JV}{RV + \varepsilon} \in [0, 1] \quad (43)$$

$$\hat{\sigma}_c = \sqrt{\max(BV, 0)} \quad (\text{continuous volatility proxy}) \quad (44)$$

Implementation: `scripts/algorithms/bipower_jump.py`. Features: `alg_bipower_variation`, `alg_jump_variation`, `alg_jump_ratio`, `alg_continuous_vol`.

Reference: Barndorff-Nielsen, O.E. & Shephard, N. (2004). Power and bipower variation with stochastic volatility and jumps. *Journal of Financial Econometrics*, 2(1).

3.5 Lee-Mykland Jump Detection (2008)

A nonparametric jump test using bipower-estimated local volatility:

$$L(t) = \frac{|r_t|}{\hat{\sigma}_{\text{BV}}(t)} \quad (45)$$

where $\hat{\sigma}_{\text{BV}}(t) = \sqrt{\frac{\pi}{2} \cdot \frac{1}{W-1} \sum_{i=1}^{W-1} |r_{t-i}| \cdot |r_{t-i-1}|}$ is the local bipower volatility over a rolling window of W ticks.

Decision: Jump detected if $L(t) > c$ (default $c = 3.0$).

Post-jump reversion:

$$\text{REV}(t) = -\frac{\log(p_t/p_J)}{r_J} \quad (46)$$

where p_J and r_J are the price and return at the last detected jump.

Implementation: `scripts/algorithms/jump_detector.py`. Features: `alg_jump_statistic`, `alg_jump_detected`, `alg_jump_magnitude`, `alg_post_jump_reversion`.

Reference: Lee, S.S. & Mykland, P.A. (2008). Jumps in financial markets: a new nonparametric test and jump dynamics. *The Review of Financial Studies*, 21(6).

4 Market Microstructure Theory

4.1 Kyle's Lambda (1985)

Measures price impact per unit of signed order flow. In Kyle's model, an informed trader's order moves the price linearly:

$$\Delta p = \lambda \cdot \text{SignedVolume} + \varepsilon \quad (47)$$

λ is estimated via OLS regression of price changes on signed trade volumes within a rolling window.

Interpretation: Higher λ = less liquid, more information per trade.

Implementation: `rust/ing/src/features/illiquidity.rs`. Windows: 100 and 500 trades. Features: `illiq_kyle_100`, `illiq_kyle_500`, `illiq_kyle_ratio`.

Reference: Kyle, A.S. (1985). Continuous auctions and insider trading. *Econometrica*, 53(6).

4.2 Amihud Illiquidity (2002)

A simpler price impact measure:

$$\text{ILLIQ} = \frac{1}{N} \sum_{i=1}^N \frac{|r_i|}{V_i} \quad (48)$$

Interpretation: Average absolute return per unit volume. Higher = less liquid.

Implementation: `rust/ing/src/features/illiquidity.rs`. Features: `illiq_amihud_100`, `illiq_amihud_500`.

Reference: Amihud, Y. (2002). Illiquidity and stock returns: cross-section and time-series effects. *Journal of Financial Markets*, 5(1).

4.3 VPIN (Volume-Synchronized Probability of Informed Trading)

Easley, López de Prado & O'Hara (2012). Estimates the probability that a trade is information-driven, using volume buckets instead of time buckets:

$$\text{VPIN} = \frac{\sum_{\tau=1}^n |V_{\tau}^B - V_{\tau}^S|}{\sum_{\tau=1}^n (V_{\tau}^B + V_{\tau}^S)} \quad (49)$$

where V_{τ}^B and V_{τ}^S are buy and sell volumes in volume bucket τ , and n is the number of buckets.

Range: $[0, 1]$. Higher = more toxic (informed) flow.

Implementation: `rust/ing/src/features/toxicity.rs`. Bucket sizes: 10 and 50. Features: `toxic_vpin_10`, `toxic_vpin_50`, `toxic_vpin_roc`.

Reference: Easley, D., López de Prado, M.M. & O'Hara, M. (2012). Flow toxicity and liquidity in a high-frequency world. *The Review of Financial Studies*, 25(5).

4.4 Bouchaud Propagator Model (2004)

Decomposes price impact into transient and permanent components:

Transient impact (power-law decay):

$$\text{TI}(t) = \frac{1}{W} \sum_{\tau=0}^{W-1} \varepsilon_{t-\tau} \cdot G(W - \tau), \quad G(\tau) = \tau^{-\alpha} \quad (50)$$

where $\varepsilon_t = \text{sign}(\text{aggressor}) \times \text{volume}$ and $\alpha = 0.5$ (Bouchaud's empirical exponent).

Permanent impact: EMA of window log-returns (long-memory drift).

Impact decay ratio:

$$\text{IDR}(t) = \frac{|\text{TI}(t)|}{|\text{PI}(t)| + \varepsilon} \quad (51)$$

Implementation: `scripts/algorithms/propagator.py`. Features: `alg_transient_impact`, `alg_permanent_impact`, `alg_impact_decay_ratio`.

Reference: Bouchaud, J.-P., Gefen, Y., Potters, M. & Wyart, M. (2004). Fluctuations and response in financial markets. *Quantitative Finance*, 4(2).

4.5 Huang-Stoll Spread Decomposition (1997)

Decomposes the bid-ask spread into adverse selection and order processing components:

$$\text{Effective Spread} = 2|p_{\text{trade}} - m_t| \quad (52)$$

$$\text{Realized Spread} = 2 \cdot d \cdot (p_{\text{trade}} - m_{t+\Delta}) \quad (53)$$

$$\text{Adverse Selection} = \text{Effective} - \text{Realized} \quad (54)$$

where $d = +1$ for buyer-initiated and -1 for seller-initiated trades, and m_t is the midprice.

Interpretation: Adverse selection component measures the permanent information content of a trade. High adverse selection $\Rightarrow$ informed trading.

Implementation: `scripts/algorithms/spread_decomp.py`. Features: `alg_adverse_component`, `alg_adverse_trend`, `alg_spread_regime`.

Reference: Huang, R.D. & Stoll, H.R. (1997). The components of the bid-ask spread. *The Review of Financial Studies*, 10(4).

4.6 Hasbrouck's Lambda (2009)

Measures permanent price impact via the information share of trades:

$$\lambda_H = \frac{\text{Var}(\text{permanent component})}{\text{Var}(\text{total price change})} \quad (55)$$

Estimated from a VAR model of trade prices and signed order flow.

Implementation: `rust/ing/src/features/illiquidity.rs`. Features: `illiq_hasbrouck_100`, `illiq_hasbrouck_500`.

Reference: Hasbrouck, J. (2009). Trading costs and returns for U.S. equities: estimating effective costs from daily data. *The Journal of Finance*, 64(3).

5 Concentration & Inequality Metrics

5.1 Herfindahl-Hirschman Index (HHI)

$$\text{HHI} = \sum_{i=1}^N s_i^2 \quad (56)$$

where $s_i = \text{OI}_i / \text{OI}_{\text{total}}$ is the market share of position i .

Range: $[1/N, 1]$. HHI = 1 for monopoly, $1/N$ for perfectly distributed. Feature: `conc_herfindahl_index`.

5.2 Gini Coefficient

$$G = \frac{\sum_{i=1}^N \sum_{j=1}^N |x_i - x_j|}{2N \sum_{i=1}^N x_i} \quad (57)$$

Range: $[0, 1]$. $G = 0$ for perfect equality, $G = 1$ for perfect inequality. Feature: `conc_gini_coefficient`.

5.3 Theil Index

An entropy-based inequality measure:

$$T = \frac{1}{N} \sum_{i=1}^N \frac{x_i}{\bar{x}} \log \left(\frac{x_i}{\bar{x}} \right) \quad (58)$$

Connection to entropy: $T = \log N - H(\mathbf{s})$, where $H(\mathbf{s})$ is the Shannon entropy of the share distribution. Maximum concentration $\Rightarrow T = \log N$; perfect equality $\Rightarrow T = 0$.

Feature: `conc_theil_index`.

Implementation: All in `rust/ing/src/features/concentration.rs`.

6 Statistical Hypothesis Testing

6.1 Correlation Testing

Pearson:

$$r_{xy} = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}} \quad (59)$$

Spearman: Pearson correlation on rank-transformed data. Captures monotonic (not just linear) relationships.

T-statistic for significance:

$$t = r \sqrt{\frac{N-2}{1-r^2}}, \quad \text{df} = N-2 \quad (60)$$

Implementation: `rust/ing/src/hypothesis/stats.rs`.

6.2 Walk-Forward Validation

Prevents lookahead bias by splitting data chronologically:

Parameter	Value
Folds	5 (expanding window)
Train/Test split	70%/30%
Success criterion	OOS correlation > threshold × IS correlation
OOS/IS ratio threshold	0.5

Overfitting detection: If OOS/IS ratio < 0.5 across folds, the signal is likely overfit.

6.3 Multiple Testing Correction

Bonferroni:

$$\alpha_{\text{corrected}} = \frac{\alpha}{m} \quad (61)$$

Controls family-wise error rate (FWER). Conservative: rejects too few when m is large.

Benjamini-Hochberg (1995):

1. Order p -values: $p_{(1)} \leq p_{(2)} \leq \dots \leq p_{(m)}$
2. Find largest k such that $p_{(k)} \leq \frac{k}{m}q$
3. Reject $H_{(1)}, \dots, H_{(k)}$

Controls false discovery rate (FDR) at level q . NAT uses $q = 0.05$.

Implementation: Agent 5-gate protocol uses BH at end of each hypothesis cycle. Config: `config/agent.toml`.

7 Online Learning Algorithms

7.1 Online Ridge Regression

Streaming linear regression with L_2 regularization, using Sherman-Morrison rank-1 updates to avoid matrix inversion at each step.

Model: $\hat{y} = \mathbf{w}^\top \mathbf{x}$, with $\mathbf{w} \in \mathbb{R}^d$.

Initialization:

$$P_0 = \frac{1}{\lambda} I_d \quad (\text{inverse regularized covariance}) \quad (62)$$

$$\mathbf{w}_0 = \mathbf{0} \quad (63)$$

Per-tick update (Sherman-Morrison):

$$P_{t+1} = P_t - \frac{P_t \mathbf{x}_t \mathbf{x}_t^\top P_t}{1 + \mathbf{x}_t^\top P_t \mathbf{x}_t} \quad (64)$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \eta \cdot (y_t - \mathbf{w}_t^\top \mathbf{x}_t) \cdot P_{t+1} \mathbf{x}_t \quad (65)$$

Complexity: $O(d^2)$ per tick (vs. $O(d^3)$ for batch inversion).

Confidence:

$$c_t = \frac{1}{1 + \mathbf{x}_t^\top P_t \mathbf{x}_t} \in (0, 1] \quad (66)$$

High confidence when the input is well-represented by the inverse covariance.

Feature importance entropy:

$$H_w = - \sum_{j=1}^d p_j \log p_j, \quad p_j = \frac{|w_j|}{\sum_k |w_k|} \quad (67)$$

High H_w = weights spread evenly (no dominant feature). Low H_w = model relies on few features.

Implementation: `scripts/algorithms/online_ridge.py`. Default: $\lambda = 1.0$, $\eta = 0.01$, update interval = 10 ticks.

Reference: Rakhlin, A. & Sridharan, K. (2013). Online learning with predictable sequences. *COLT*.

8 Regime Detection & Classification

8.1 Gaussian Mixture Model (GMM)

Offline-trained probabilistic classifier. The density of observation $\mathbf{x}$ is:

$$p(\mathbf{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \quad (68)$$

NAT parameterization: $K = 5$ components, 5D feature space:

1. Kyle's lambda (illiquidity)
2. VPIN (toxicity)
3. Absorption z-score (accumulation/distribution)
4. Hurst exponent (trending vs. mean-reverting)
5. Whale net flow (institutional positioning)

Regime labels: Accumulation, Markup, Distribution, Markdown, Ranging (Wyckoff theory).

Training: sklearn `GaussianMixture` on historical data with `StandardScaler`. **Inference:** Rust implementation in `rust/ing/src/ml/regime.rs` for online scoring.

8.2 Absorption Ratio

Measures volume absorption relative to price movement:

$$\text{AR}(w) = \frac{\sum_{t \in w} V_t}{|\Delta p_w| + \varepsilon} \quad (69)$$

High AR = volume being absorbed without moving price (accumulation/distribution). Reported as z-score vs. rolling history.

Implementation: `rust/ing/src/features/regime/absorption.rs`.

8.3 Regime Composite Score

Weighted combination of regime indicators:

$$\text{RC} = 0.35 \cdot \text{absorption} + 0.30 \cdot \text{divergence} + 0.15 \cdot \text{churn} + 0.20 \cdot \text{range_position} \quad (70)$$

Implementation: `rust/ing/src/features/regime/composite.rs`.

9 Trend & Time Series Analysis

9.1 Hurst Exponent

Measures long-range dependence via rescaled range (R/S) analysis:

$$\frac{R(n)}{S(n)} \sim C \cdot n^H \quad \text{as } n \rightarrow \infty \quad (71)$$

Interpretation:

$H > 0.5$	Trending (persistent)
$H = 0.5$	Random walk
$H < 0.5$	Mean-reverting (anti-persistent)

Implementation: `rust/ing/src/features/trend.rs`. Windows: 300, 600 ticks. Features: `trend_hurst_300`, `trend_hurst_600`.

Reference: Mandelbrot, B.B. (1971). When can price be arbitrated efficiently? A limit to the validity of the random walk and martingale models. *The Review of Economics and Statistics*.

9.2 Momentum (Linear Regression Slope)

$$\hat{\beta} = \frac{\sum_{i=1}^N (t_i - \bar{t})(p_i - \bar{p})}{\sum_{i=1}^N (t_i - \bar{t})^2} \quad (72)$$

R-squared:

$$R^2 = 1 - \frac{\sum (p_i - \hat{p}_i)^2}{\sum (p_i - \bar{p})^2} \quad (73)$$

High R^2 with positive slope = strong uptrend. High R^2 with negative slope = strong downtrend.

Implementation: `rust/ing/src/features/trend.rs`. Windows: 60, 300, 600 ticks.

9.3 EMA Crossover

Short-term EMA crossing long-term EMA as a trend signal:

$$\text{EMA}_\alpha(t) = \alpha \cdot x_t + (1 - \alpha) \cdot \text{EMA}_\alpha(t - 1), \quad \alpha = \frac{2}{N + 1} \quad (74)$$

Crossover signal: $\text{EMA}_{10} - \text{EMA}_{50}$. Positive = bullish, negative = bearish.

Implementation: `rust/ing/src/features/trend.rs`. Features: `ma_crossover`, `ma_crossover_norm`.

10 Complete Algorithm Inventory

10.1 Implemented Streaming Algorithms (26)

All algorithms implement the `MicrostructureAlgorithm` ABC from `scripts/algorithms/base.py`, with a `step()` method called at each tick (100ms).

#	Algorithm	Mathematical Basis	Features
1	Kalman Imbalance	OU Kalman filter	4
2	Multi-Level Imbalance	Weighted depth composite	3
3	Regime-Gated Imbalance	Percentile gating	3
4	Online Ridge	Sherman-Morrison updates	3
5	Jump Detector	Lee-Mykland (2008)	4
6	Bipower Jump	BNS (2004) decomposition	4
7	VPIN Regime	Toxicity gating	3
8	Switching OU	Hamilton filter + dual Kalman	4
9	Entropy Momentum	Conditional momentum	3
10	OI Divergence	Price-OI confirmation	3
11	Surprise Signal	Entropy transition detection	3
12	Optimal Entry	SPRT + Kalman	3
13	Propagator	Bouchaud impact model	3
14	Spread Decomposition	Huang-Stoll (1997)	3
15	Weighted OFI	Depth-decay kernel	3
16	Trade-Through	Queue depletion model	3
17	Hawkes Intensity	Self-exciting process	4
18	Funding Reversion	Mean-reversion on funding	3

10.2 Algorithms Not Yet Implemented (High-Value Candidates)

Algorithm	Mathematical Basis	Value Proposition
FIR Filter	$y[n] = \sum_{k=0}^M b_k x[n-k]$	Linear-phase frequency-domain feature extraction
IIR Filter	$y[n] = \sum_{k=0}^M b_k x[n-k] - \sum_{k=1}^N a_k y[n-k]$	Efficient recursive filtering with fewer coefficients
Butterworth	Maximally flat magnitude response	Clean low/high/band-pass of feature signals
Online PCA	Rank-1 covariance updates	Streaming dimensionality reduction
Full HMM	Forward-backward, Baum-Welch EM	Multi-step regime forecasting
Two-Scales RV	Zhang et al. (2005)	Microstructure noise correction
Realized GARCH	Conditional volatility	Volatility forecasting
Mixture of Experts	Exponential weights algorithm	Regime-adaptive feature combination
Particle Filter	Sequential Monte Carlo	Nonlinear/non-Gaussian state estimation

Algorithm	Mathematical Basis	Value Proposition
Wavelet Transform	Multi-resolution analysis	Time-frequency decomposition

11 Autonomous Feature Engineering Agent

11.1 Current Agent Architecture

NAT’s existing agent framework (`scripts/agent/daemon.py`) implements a 5-gate hypothesis testing protocol:

MANIFEST -> GENERATE -> EXECUTE -> FDR -> MONITOR -> SLEEP

5-gate protocol per hypothesis:

1. **Discovery gate:** $IC > \text{threshold}$, ΔIC significant
2. **Cost gate:** $MI > I_{\min}(\text{fee}, \sigma_r)$
3. **Temporal replication:** Signal persists across time windows
4. **Symbol replication:** Signal generalizes across assets
5. **Correlation dedup:** Not redundant with existing features ($\rho < 0.7$)

FDR control (Benjamini-Hochberg, $q = 0.05$) at end of each cycle.

11.2 Proposed Extension: Feature Factory Agent

11.2.1 Generator Operations (Feature Algebra)

Define a typed algebra of feature transformations:

Operation	Definition	Type
$\text{diff}(f, n)$	$\Delta^n f_t = f_t - f_{t-n}$	$\mathbb{R} \rightarrow \mathbb{R}$
$\text{zscore}(f, w)$	$(f_t - \bar{f}_w) / \sigma_{f,w}$	$\mathbb{R} \rightarrow \mathbb{R}$
$\text{ema}(f, \alpha)$	$\alpha f_t + (1 - \alpha) \text{ema}_{t-1}$	$\mathbb{R} \rightarrow \mathbb{R}$
$\text{kalman}(f, \theta)$	OU Kalman filter with speed θ	$\mathbb{R} \rightarrow \mathbb{R}^4$
$\text{fir}(f, \mathbf{b})$	$\sum_{k=0}^M b_k f_{t-k}$	$\mathbb{R} \rightarrow \mathbb{R}$
$\text{iir}(f, \mathbf{a}, \mathbf{b})$	$\sum b_k f_{t-k} - \sum a_k g_{t-k}$	$\mathbb{R} \rightarrow \mathbb{R}$
$\text{rank}(f, w)$	Percentile rank in window w	$\mathbb{R} \rightarrow [0, 1]$
$\text{sign}(f)$	$\text{sgn}(f_t)$	$\mathbb{R} \rightarrow \{-1, 0, 1\}$
$\text{interact}(f_1, f_2)$	$f_{1,t} \cdot f_{2,t}$	$\mathbb{R}^2 \rightarrow \mathbb{R}$
$\text{ratio}(f_1, f_2)$	$f_{1,t} / (f_{2,t} + \varepsilon)$	$\mathbb{R}^2 \rightarrow \mathbb{R}$
$\text{residual}(f_1, f_2)$	$f_{1,t} - \hat{\beta} f_{2,t}$	$\mathbb{R}^2 \rightarrow \mathbb{R}$

11.2.2 Evaluation Metrics

For each candidate feature f , compute against forward returns r_{t+h} at horizons $h \in \{1s, 5s, 15s, 1m, 5m\}$:

Metric	Definition	Threshold
IC	$\text{Spearman}(f_t, r_{t+h})$	$ IC > 0.02$
ICIR	$\text{mean}(IC_w) / \text{std}(IC_w)$ across windows	$ICIR > 0.5$
MI	$\hat{I}_{\text{KSG}}(f; r_{t+h})$	$MI > I_{\min}$
Marginal MI	$I(f; r \mathbf{z}_{\text{existing}})$ via CMI	> 0
Turnover	$\text{autocorr}(f_t, 1)$	> 0.8 (low churn)
Max corr	$\max_j \rho(f, z_j) $	< 0.7 (novelty)

11.2.3 Selection Protocol (Extended 5-Gate)

1. **IC significance:** t -test on IC, $p < 0.01$
2. **Cost viability:** $MI > I_{\min}(\text{fee}, \sigma_r)$
3. **Temporal stability:** ICIR > 0.5 across ≥ 3 non-overlapping windows
4. **Novelty:** Max correlation with existing feature set < 0.7
5. **FDR control:** Benjamini-Hochberg at $q = 0.05$ across batch

11.2.4 Evolution Step (Missing Piece)

The gap between the current generate-test-prune system and full autonomy is the **evolution step**:

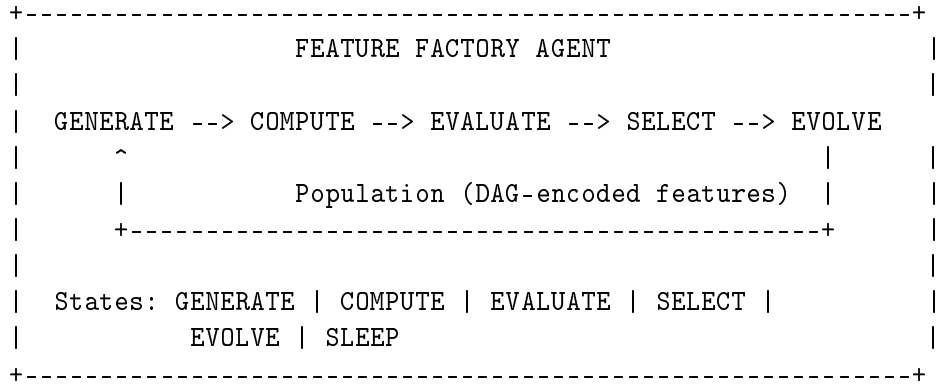
Crossover: Given two surviving features $f_A = \text{kalman}(\text{imbalance}, 0.1)$ and $f_B = \text{ema}(\text{entropy}, 0.05)$,
generate: $f_{AB} = \text{kalman}(\text{imbalance} \times \text{entropy}, 0.1)$

Mutation: Perturb parameters of survivors: $\theta \rightarrow \theta \cdot (1 + \mathcal{N}(0, 0.1))$, $w \rightarrow w + \text{Uniform}(-5, 5)$

Population management: Keep top- N by ICIR. Kill bottom quartile. Fill with crossover + mutation + fresh generation.

Fitness landscape: Track which operation types and parameter ranges produce survivors $\Rightarrow$ bias future generation toward productive regions.

11.2.5 Architecture



Features are encoded as serializable DAGs (directed acyclic graphs) of operations, enabling reconstruction from raw inputs without storing computed values.

11.2.6 Progression: Semi-Autonomous to Autonomous

Phase	Human Role	Agent Role
Phase 1 (Semi)	Define operation set, review survivors, approve promotions	Generate, evaluate, rank, report
Phase 2 (Guided)	Set meta-parameters, monitor population health	Generate, evaluate, select, evolve within bounds
Phase 3 (Auto)	Monitor dashboards, intervene on anomalies	Full loop: generate, evaluate, select, evolve, deploy

12 Signal Processing: FIR, IIR & Advanced Filtering

This section covers the mathematical foundations for filters not yet implemented in NAT but identified as high-value additions for the Feature Factory Agent.

12.1 Finite Impulse Response (FIR) Filters

A causal FIR filter of order M :

$$y[n] = \sum_{k=0}^M b_k x[n-k] \quad (75)$$

Properties:

- Always stable (no feedback)
- Can achieve exact linear phase (symmetric coefficients)
- Requires more coefficients than IIR for sharp transitions

Transfer function (z -transform):

$$H(z) = \sum_{k=0}^M b_k z^{-k} \quad (76)$$

Design methods:

- **Window method:** Truncate ideal impulse response, apply window (Hamming, Blackman, Kaiser)
- **Parks-McClellan:** Optimal equiripple design (minimax criterion)
- **Least-squares:** Minimize $\int |H(e^{j\omega}) - H_d(e^{j\omega})|^2 d\omega$

Application to NAT: Low-pass filtering of noisy features (e.g., imbalance, flow intensity) to extract slow-moving components. Band-pass to isolate specific frequency bands of interest.

12.2 Infinite Impulse Response (IIR) Filters

A causal IIR filter:

$$y[n] = \sum_{k=0}^M b_k x[n-k] - \sum_{k=1}^N a_k y[n-k] \quad (77)$$

Transfer function:

$$H(z) = \frac{\sum_{k=0}^M b_k z^{-k}}{1 + \sum_{k=1}^N a_k z^{-k}} = \frac{B(z)}{A(z)} \quad (78)$$

Stability condition: All poles of $A(z)$ must lie inside the unit circle $|z| < 1$.

Properties:

- More efficient than FIR (fewer coefficients for same sharpness)
- Cannot achieve linear phase in general
- Can be unstable if poorly designed
- The EMA is the simplest IIR: $y[n] = \alpha x[n] + (1 - \alpha)y[n-1]$, i.e., $b_0 = \alpha$, $a_1 = -(1 - \alpha)$

12.3 Butterworth Filter

Maximally flat magnitude response in the passband:

$$|H(j\omega)|^2 = \frac{1}{1 + (\omega/\omega_c)^{2N}} \quad (79)$$

where N is the filter order and ω_c is the cutoff frequency.

Properties:

- No ripple in passband or stopband
- Roll-off: $-20N$ dB/decade
- Monotonically decreasing magnitude response

Application to NAT: Clean separation of “fast” and “slow” components of microstructure features, parameterized by cutoff frequency that the Feature Factory Agent can optimize.

12.4 Relationship to Existing NAT Filters

NAT Filter	Type	Equivalent
EMA	IIR (1st order)	$b_0 = \alpha, a_1 = \alpha - 1$
Kalman (OU)	Optimal (state-space)	Time-varying IIR with gain K_t
Rolling mean	FIR (uniform)	$b_k = 1/N$ for $k = 0, \dots, N - 1$

The Kalman filter subsumes fixed-coefficient filters by adapting its gain K_t based on the innovation covariance. FIR/IIR filters are useful when the adaptation is unnecessary or when specific frequency-domain properties (e.g., linear phase) are required.

13 NEAR Protocol: Technical Analysis

13.1 Architecture

Component	Description
Consensus	PoS with threshold-based block producer selection
Block time	600ms, ~1.2s finality
Sharding	Nightshade — state split across shards processed in parallel
Runtime	Wasm-based VM (contracts compile from Rust or JS)
Core client	nearcore (Rust)
Shards (current)	8 (benchmarked at 1M+ TPS across 70 shards)

Nightshade 2.0 (2024–2025): Stateless validation — validators verify via cryptographic proofs without storing full shard state. **Nightshade 3.0 (Feb 2026):** Privacy shards running inside TEEs.

13.2 Strengths vs. Other L1s

Dimension	NEAR	Comparison
Scaling	Horizontal (sharding)	Solana: monolithic, higher HW reqs
Finality	~1.2s	Solana: ~400ms (faster)
Fees	~\$0.001	Ethereum: \$1–10+ (L1)
Accounts	Human-readable names	Ethereum: hex addresses
Chain abstraction	Native (Intents, \$10B volume)	Unique — no other L1 has this
AI agents	747+ active, TEE runtime	Most developed on-chain AI infra

13.3 Relevance for Quantitative Trading

- **On-chain orderbooks:** Orderly Network (shared orderbook), Tonic (CLOB DEX) — 600ms blocks make on-chain market making feasible
- **Caveat:** DeFi TVL and volume still thin vs. Ethereum/Solana — liquidity is the constraint, not technology
- **AI agent runtime:** IronClaw (TEE-isolated execution), confidential GPU marketplace

13.4 Potential Contribution Areas

Given an embedded systems + quant + Rust profile:

1. **nearcore performance:** Sharding runtime optimization, parallel execution scheduling
2. **On-chain orderbook infra:** CLOB DEX smart contracts — microstructure knowledge is rare in crypto
3. **AI agent execution runtime:** TEE integration, deterministic scheduling
4. **MPC/Chain Signatures:** Low-level Rust cryptographic infrastructure

14 Hermes Agents & Evolutionary Self-Learning

14.1 Hermes Agent (Nous Research)

An open-source autonomous AI *runtime* (not a model) released Feb 2026. Core mechanism: after any task with 5+ tool calls, a background process distills the trajectory into a Markdown skill file. Over time, it accumulates a library of reusable procedures.

Capabilities:

- Persistent memory across sessions
- Autonomous skill creation from trajectories
- Scheduled automations
- MCP server integration for tool extensibility

Limitation: Learns *procedures* (how to do X), not *strategies* (what X to try). No hypothesis generation, no statistical testing, no evolutionary selection.

14.2 Comparison with NAT Agent

Capability	Hermes	NAT Agent
Hypothesis generation	–	✓
Statistical testing (5-gate)	–	✓
FDR control	–	✓
Cost viability gating	–	✓
Skill/procedure learning	✓	–
Cross-agent coordination	–	✓ (MetaAgent)
Evolutionary selection	–	Partial (prune, no crossover)

14.3 Related Evolutionary Approaches

QuantEvolve (arXiv 2510.18569): Multi-agent evolutionary framework for trading strategy space exploration. Hypothesis-driven search inspired by AlphaEvolve.

Vectorial GP (arXiv 2504.05418): Genetic programming evolving trading rules from technical indicators with multi-objective optimization (return vs. risk).

NNGPT (arXiv 2511.20333): LLM as self-improving AutoML engine — generate pipeline, assess, refine in closed loop.

14.4 The Generate-Test-Prune-Evolve Loop

NAT’s current architecture implements three of four phases:

GENERATE	----->	TEST	----->	PRUNE	----->	[EVOLVE]	----->	GENERATE
Hypothesis generators		5-gate protocol		FDR + correlation dedup		Missing: crossover, mutation		

The missing evolution step is the key upgrade: crossover and mutation of surviving strategies to spawn new candidates, rather than generating from scratch each cycle. This transforms the system from a **search engine** (enumerate and filter) into an **evolutionary optimizer** (breed and improve).

15 Evolutionary & AutoML Frameworks for Strategy Discovery

This section summarizes three recent papers that are closest to the vision of an autonomous, self-improving strategy generation system. We first introduce the foundational concepts — evolutionary computation, genetic programming, and AutoML — with worked examples before diving into each paper.

15.1 Foundations: Evolutionary Computation

Evolutionary computation borrows from biological evolution: a *population* of candidate solutions is maintained, evaluated for *fitness*, and iteratively improved through *selection*, *crossover* (recombination), and *mutation*. The process is domain-agnostic — what changes is how solutions are represented and how fitness is measured.

15.1.1 Core Loop

1. INITIALIZE: Create N random candidate solutions (population)
2. EVALUATE: Score each candidate with a fitness function
3. SELECT: Choose parents (higher fitness = higher probability)
4. CROSSOVER: Combine two parents to produce offspring
5. MUTATE: Randomly perturb offspring
6. REPLACE: Offspring replace worst members of population
7. REPEAT from step 2 until convergence or budget exhausted

15.1.2 Example: Evolving a Trading Signal Weight Vector

Suppose we want to find optimal weights $\mathbf{w} = (w_1, w_2, w_3)$ for a composite signal:

$$\text{signal}(t) = w_1 \cdot \text{imbalance}(t) + w_2 \cdot \text{entropy}(t) + w_3 \cdot \text{VPIN}(t) \quad (80)$$

Step 1 — Initialize: Create 100 random weight vectors, e.g.:

$$\begin{aligned} \mathbf{w}_1 &= (0.3, -0.1, 0.5) \\ \mathbf{w}_2 &= (-0.2, 0.8, 0.1) \\ &\vdots \end{aligned}$$

Step 2 — Evaluate: Backtest each on historical data, compute Sharpe ratio as fitness:

$$\begin{aligned} f(\mathbf{w}_1) &= 1.2 \quad (\text{decent}) \\ f(\mathbf{w}_2) &= -0.3 \quad (\text{losing money}) \end{aligned}$$

Step 3 — Select: Tournament selection picks k random candidates, takes the best. With tournament size $k = 5$: draw 5 random candidates, the one with highest Sharpe becomes a parent. Repeat for second parent.

Step 4 — Crossover: Blend two parents. Single-point crossover at position 2:

$$\begin{aligned} \text{Parent A} &= (\underbrace{0.3, -0.1}_{\text{keep}}, \underbrace{0.5}_{\text{swap}}) \\ \text{Parent B} &= (\underbrace{-0.2, 0.8}_{\text{swap}}, \underbrace{0.1}_{\text{keep}}) \\ \text{Child} &= (0.3, -0.1, 0.1) \quad (\text{A's head} + \text{B's tail}) \end{aligned}$$

Step 5 — Mutate: Add Gaussian noise to one gene with small probability p_m :

$$w'_2 = w_2 + \mathcal{N}(0, 0.05) = -0.1 + 0.03 = -0.07 \quad (81)$$

Step 6 — Replace: The child $(0.3, -0.07, 0.1)$ replaces the worst member of the population.

After 50–200 generations, the population converges toward high-Sharpe weight vectors. The key advantage over grid search: crossover *recombines* partial solutions that work, so the search is directed, not random.

15.1.3 Selection Methods

Method	How It Works
Tournament	Draw k random candidates, pick the best. Larger k = stronger selection pressure.
Roulette Wheel	Probability proportional to fitness: $P(i) = f_i / \sum_j f_j$. Sensitive to fitness scaling.
Rank-Based	Sort by fitness, assign selection probability by rank (not raw fitness). More robust.
Elitism	Top e individuals survive unchanged to next generation (prevents losing the best).

15.1.4 Quality-Diversity (MAP-Elites)

Standard evolution converges to a single optimum. **MAP-Elites** (Mouret & Clune, 2015) instead maintains a *grid* of solutions, one per cell, where each cell represents a different *behavioral niche*. A new solution enters a cell only if it outperforms the current occupant.

Example in trading: Define a 2D grid with axes “trading frequency” (low/medium/high) and “max drawdown” ($<5\%$ / $5\text{--}15\%$ / $>15\%$). This gives $3 \times 3 = 9$ cells. Evolution fills each cell with the best strategy *for that niche*. You get 9 diverse strategies, not 9 copies of the same one.

		Max Drawdown			
		<5%	5-15%	>15%	
Low freq		+-----+-----+-----+			<-- conservative
	SR=0.8	SR=1.1	SR=1.9		
Med freq		+-----+-----+-----+			
	SR=0.5	SR=1.3	SR=2.1		
High freq		+-----+-----+-----+			<-- aggressive
	SR=0.3	SR=0.9	SR=1.5		
		+-----+-----+-----+			

Each cell answers a different question: “What’s the best strategy for an investor who wants low drawdown and trades infrequently?” This is the core of QuantEvolve’s archive.

15.2 Foundations: Genetic Programming (GP)

While standard evolutionary algorithms evolve fixed-length vectors (weights, parameters), **genetic programming** evolves *programs* — typically represented as expression trees.

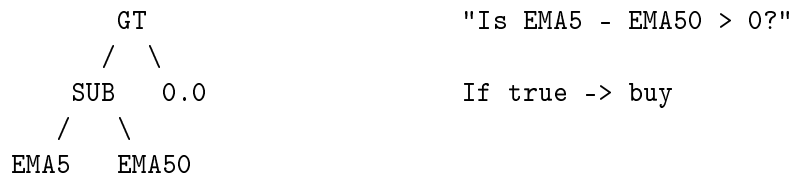
15.2.1 Tree Representation

A trading rule is a tree where:

- **Leaves (terminals):** Market data (price, volume, EMA, RSI, imbalance)

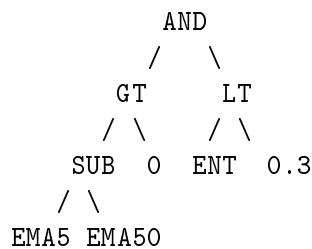
- **Internal nodes (functions):** Mathematical operations (ADD, MULT, SUB, GT, AND)
- **Root:** Produces the final output (signal value or buy/sell decision)

Example tree (a momentum crossover rule):



This tree computes: $GT(EMA_5 - EMA_{50}, 0.0)$, returning 1 (buy) if short EMA exceeds long EMA.

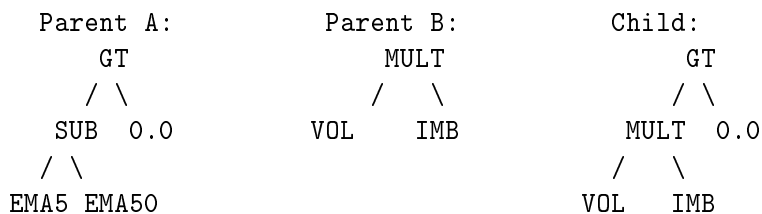
A more complex tree (entropy-gated momentum):



This computes: $(EMA_5 - EMA_{50} > 0) \text{ AND } (\text{entropy} < 0.3)$. Buy only when momentum is positive *and* entropy is low (structured market). This is the kind of conditional logic that GP discovers automatically.

15.2.2 GP Crossover

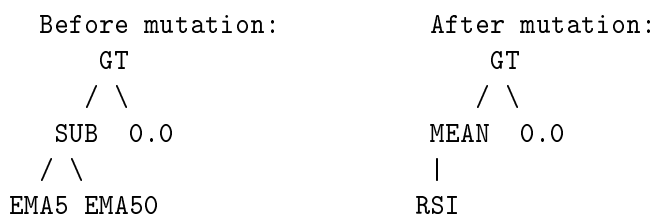
Select a random subtree from each parent, swap them:



The SUB subtree in Parent A is replaced by the MULT subtree from Parent B. The child now computes $GT(VOL \times IMB, 0.0)$ — a volume-imbalance signal that neither parent had.

15.2.3 GP Mutation

Replace a random subtree with a new random subtree:



The momentum rule mutated into a mean-reversion rule (buy when mean RSI > 0). Most mutations are harmful (lower fitness), but occasionally one discovers a better structure.

15.2.4 Bloat Control

GP trees tend to grow without bound (“bloat”). Controls:

- **Max depth:** Trees deeper than $D_{\max}$ are rejected (Vectorial GP uses $D_{\max} = 13$)
- **Max nodes:** Cap total nodes (Vectorial GP uses 90)
- **Parsimony pressure:** Penalize fitness by tree size: $f' = f - c \cdot |\text{nodes}|$

15.2.5 Vectorial GP: The Key Innovation

Standard GP operates on *scalars*: each terminal is a single number (today’s close, current RSI). Vectorial GP operates on *vectors*: each terminal is a window of values (last 21 closes, last 21 RSI values).

Scalar GP example:

```
SUB(Close_today, EMA50_today) = 105.3 - 103.1 = 2.2
```

Vectorial GP example:

```
DOT(Close[t-20:t], Volume[t-20:t])
= sum of element-wise products over 21 days
= captures price-volume correlation over the lookback window
```

The vector DOT product automatically computes a correlation-like feature that a scalar GP cannot express without hand-engineering 21 separate lagged terminals. This is why VGP outperforms standard GP.

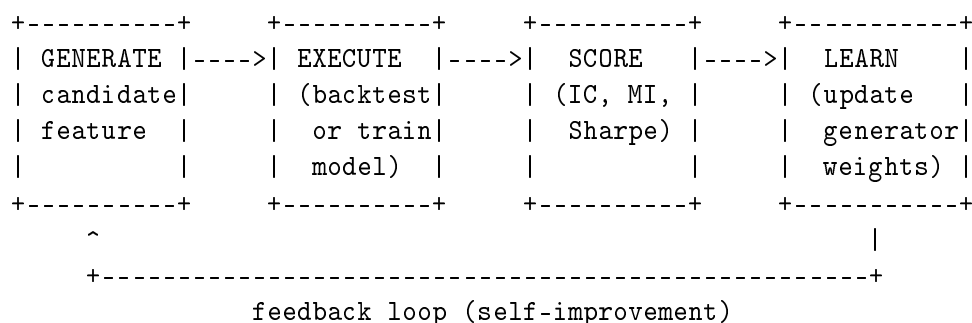
Strongly-Typed GP (STVGP): Forces the tree’s root to output a boolean (buy/sell), not a number. This eliminates the arbitrary threshold problem (“buy when signal > 1.0” — why 1.0?). The type system ensures:

- Data nodes (OHLCV, indicators) produce vectors
- Relational nodes (GT, LT) consume vectors, produce booleans
- Logic nodes (AND, OR, IF_ELSE) consume and produce booleans
- Root must be relational or boolean

15.3 Foundations: AutoML and Self-Improving Systems

AutoML automates the machine learning pipeline: feature engineering, model selection, hyperparameter tuning. Traditional AutoML (Optuna, Auto-sklearn) uses **search** — try thousands of configurations, pick the best. NNGPT replaces search with **generation** — an LLM proposes configurations directly, then learns from outcomes.

15.3.1 The Closed-Loop Self-Improvement Pattern



Without learning (NAT today): Generator produces candidates from a fixed rule set. Bad candidates are filtered (5-gate), but the generator doesn’t improve. Generation 100 is no smarter than generation 1.

With learning (NNGPT pattern): Generator tracks which operation types produce survivors. After 50 cycles, it has learned: “Kalman filters on imbalance features survive $3\times$ more often than raw EMA. Interaction terms between entropy and flow survive $2\times$ more often than single-feature transforms.” Generation 100 produces better candidates than generation 1 because the generator has been *fine-tuned on its own execution history*.

15.3.2 Concrete Example: Feature Factory Learning

Cycle 1 (no learning): Generator produces 100 random features:

- 20 EMA variants $\rightarrow$ 2 survive gates (10% survival)
- 20 z-score variants $\rightarrow$ 3 survive (15%)
- 20 Kalman variants $\rightarrow$ 6 survive (30%)
- 20 interaction terms $\rightarrow$ 4 survive (20%)
- 20 ratio features $\rightarrow$ 1 survives (5%)

Cycle 2 (with learning): Generator observes survival rates and adjusts generation probabilities:

- Kalman variants: 30% $\rightarrow$ generate 40 (up from 20)
- Interaction terms: 20% $\rightarrow$ generate 25
- Z-score variants: 15% $\rightarrow$ generate 20
- EMA variants: 10% $\rightarrow$ generate 10 (down)
- Ratio features: 5% $\rightarrow$ generate 5 (down)

Over time, the generator converges toward the productive region of the feature space. This is the “LoRA fine-tuning” of NNGPT, adapted to feature engineering.

15.3.3 Early-Stop Predictor

Training a model or backtesting a feature is expensive. An early-stop predictor estimates final quality from partial results:

Example: After evaluating a candidate feature on only the first 2 hours of data (out of 24 hours total):

- If ICIR after 2h < 0.1 : predict final ICIR < 0.2 with 80% confidence $\Rightarrow$ **kill early**, save 22 hours of compute
- If ICIR after 2h > 0.5 : predict final ICIR $> 0.3 \Rightarrow$ continue full evaluation

NNGPT trains this predictor on historical (early-metric, final-metric) pairs. In NAT, the equivalent is: collect (IC-after-1-window, IC-after-all-windows) pairs from past feature evaluations, train a simple regression to predict final IC from early IC.

15.3.4 RL Reward Shaping for Feature Generation

Reinforcement learning reward for a candidate feature f :

$$r(f) = \underbrace{w_1 \cdot \mathbb{I}[\text{computes without error}]}_{\text{validity}} + \underbrace{w_2 \cdot \text{IC}(f)}_{\text{predictiveness}} + \underbrace{w_3 \cdot \mathbb{I}[\text{MI} > I_{\min}]}_{\text{cost viability}} + \underbrace{w_4 \cdot (1 - \max_j |\rho(f, z_j)|)}_{\text{novelty}} \quad (82)$$

Example scoring:

Feature	Valid	IC	Cost OK	Novelty	Total
kalman(imb, 0.1)	1.0	0.04	1.0	0.8	2.84
ema(entropy, 50)	1.0	0.01	0.0	0.9	1.91
ratio(vol, 0)	0.0	—	—	—	0.00
zscore(imb $\times$ ent, 100)	1.0	0.06	1.0	0.7	2.76

The RL loop pushes the generator toward features like “kalman(imb, 0.1)” (high reward) and away from features like “ratio(vol, 0)” (crashes) or “ema(entropy, 50)” (doesn’t cover costs).

15.4 QuantEvolve: Multi-Agent Evolutionary Strategy Discovery

Paper: Yun, J., Lee, H.J. & Jeon, I. (2025). QuantEvolve: Automating Quantitative Strategy Discovery through Multi-Agent Evolutionary Framework. *arXiv:2510.18569*. Accepted for oral presentation at 2nd Workshop on LLMs and Generative AI for Finance (AI4F), ACM ICAIF 2025.

15.4.1 Architecture

A quality-diversity evolutionary system (MAP-Elites variant) where LLM agents evolve trading strategies represented as **executable Python code**. Multiple islands evolve populations independently, sharing a common feature map archive.

Four specialized agents per generation cycle:

1. **Data Agent** — analyzes input data schema, identifies viable strategy categories
2. **Research Agent** — generates hypotheses from parent/cousin strategies plus accumulated insights
3. **Coding Team** — implements hypotheses as Python with `initialize(context)` and `handle_data(context, data)`, backtests via Zipline, iteratively debugs
4. **Evaluation Team** — extracts actionable insights, categorizes results

LLM backbone: Qwen3-30B-A3B (lightweight) and Qwen3-80B-A3B (reasoning).

15.4.2 Feature Map Archive (MAP-Elites)

Strategies are binned along 6 dimensions: strategy category (binary-encoded, e.g., “101” = momentum + mean-reversion), trading frequency, max drawdown, Sharpe ratio, Sortino ratio, total return. Continuous dimensions are discretized into equal bins over observed min-max. Each cell retains only the best strategy; displaced strategies go to a separate evolutionary database.

15.4.3 Evolution Operators

Parent selection balances exploitation vs. exploration:

$$P(s_p = s) = \begin{cases} \alpha/|M_I| & \text{if } s \in M_I \text{ (feature map)} \\ (1 - \alpha)/|I| & \text{if } s \in I \text{ (full island)} \end{cases} \quad (83)$$

with $\alpha = 0.5$.

Cousin sampling uses three strategies: best performers, feature-space neighbors (perturbed via $f_c^d = \lfloor \mathcal{N}(f_p^d, \sigma_d^2) \rfloor$ for continuous dims, BitFlip with $k_{bf} = n/4$ for categorical), and uniform random.

Migration: Top 10% exchanged across islands every M generations.

Insight curation: Every $K = 50$ generations, accumulated insights are deduplicated and pruned.

15.4.4 Fitness Function

Equal-weighted composite:

$$\text{Score} = \text{SR} + \text{IR} + \text{MDD} \quad (84)$$

where SR = Sharpe ratio, IR = information ratio vs. market-cap benchmark, MDD = maximum drawdown. This balances absolute risk-adjusted return, relative outperformance, and tail risk.

15.4.5 Key Results

On US equities (AAPL, NVDA, AMZN, GOOGL, MSFT, TSLA; test Aug 2022–Jul 2025): generation-150 strategies achieved SR 1.52, cumulative return 256%, MDD -32% , vs. best baseline Risk Parity at SR 1.22, CR 130%. Transaction costs modeled at \$0.0075/share with quadratic slippage.

15.4.6 Relation to AlphaEvolve

Builds on the AlphaEvolve (Novikov et al.) and AI Scientist (Lu et al.) paradigm of LLM-driven evolutionary search over code. Key distinction: QuantEvolve adds a **quality-diversity archive** (MAP-Elites feature map) to maintain strategy diversity across investor preferences, rather than optimizing a single objective.

15.4.7 Limitations

- No formal statistical correction for data snooping during the search process
- Generated hypotheses may be post-hoc rationalizations rather than causal mechanisms
- Each cycle requires 5–10 LLM inferences, limiting scalability
- Evaluated on only 6 equities + 2 futures with daily OHLCV; baselines are not SOTA
- No out-of-distribution or regime-shift stress testing

15.4.8 Relevance to NAT

QuantEvolve’s MAP-Elites archive maps directly to NAT’s need: maintain a *diverse* population of features/strategies across different market regimes and frequencies, rather than converging on a single best. The multi-agent structure (Data $\rightarrow$ Research $\rightarrow$ Code $\rightarrow$ Evaluate) parallels NAT’s existing MANIFEST $\rightarrow$ GENERATE $\rightarrow$ EXECUTE $\rightarrow$ FDR pipeline. The key takeaway is the **quality-diversity** principle: optimize not just fitness but coverage of the strategy space.

15.5 Vectorial Genetic Programming for Trading Rules

Paper: Menoita, R. & Silva, S. (2025). Evolving Financial Trading Strategies with Vectorial Genetic Programming. *arXiv:2504.05418*. Published at GECCO 2025 Companion (ACM).

15.5.1 Vectorial GP vs. Standard GP

Standard GP operates on scalar terminals and produces scalar outputs. Vectorial GP (VGP) extends this by allowing **fixed-length vector inputs** (window of 21 trading days) and introducing vector-native operators. All vectors are either size 1 (scalar) or size 21. When a VGP tree produces a vector output, the mean is taken and thresholded for signal generation.

Key insight: Vectors naturally encode temporal context (a lookback window) directly in the GP representation, eliminating the need to hand-engineer lagged features.

15.5.2 Terminal and Function Sets

Terminals (13): OHLCV, four EMAs (periods 5, 13, 50, 200), RSI(14), profit percentage, smallEmaDiff (EMA5 – EMA13), bigEmaDiff (EMA50 – EMA200).

Standard GP functions: ADD, MULT, SUB, DIV (protected), NEG, SIN, COS, TAN, SIGNUM, GT.

VGP additions: DOT, MEAN, STD_VAR, CUM_MEAN, GT_THAN (compares vector means).

15.5.3 Three Variants

VGP: Basic vectorial GP with vector operators.

CVGP (Complex VGP): Vectors contain complex elements; adds LOG, SQRT, complex trigonometric functions. Only the real part of the output drives trading signals. Explores a richer function space.

STVGP (Strongly-Typed VGP): Enforces a type system with three categories — vectorial functions (data manipulation), relational functions (vectors/booleans $\rightarrow$ booleans), boolean functions (AND, OR, XOR, NOT, IF_ELSE). The root node must be relational or boolean, so the tree outputs a boolean directly: true = buy, false = sell. **Eliminates the arbitrary scalar threshold.**

15.5.4 Fitness Function

Single-objective optimization with tournament selection (not multi-objective):

$$f = \begin{cases} \text{ROI} \times w_{\text{rate}} & \text{if } \text{ROI} > 0 \\ \text{ROI} & \text{if } \text{ROI} \leq 0 \end{cases} \quad (85)$$

where $\text{ROI} = 100 \times \text{TotalProfit}/M$, $M = 1000$ (fixed investment per trade), and w_{rate} is the win rate (fraction of profitable trades). The win-rate multiplier rewards consistency only when profitable.

15.5.5 Overfitting Mitigation

Training data split into three equal segments; each generation evaluates on only one segment (rotating). Every 50th “super generation” evaluates on all three. A 100-day buffer at the start is skipped, and the first evaluation day is randomized per generation.

15.5.6 GP Parameters

Parameter	Value
Population	3,000
Generations	50
Tournament size	30
Mutation rate	0.001
Max tree depth	13
Max nodes	90
Runs per config	10

15.5.7 Key Results

Kruskal-Wallis tests ($p < 0.05$): on KO (Coca-Cola), all three VGP variants significantly outperformed standard GP ($p < 0.005$). Critical difference analysis across all instruments: **STVGP consistently ranked best; standard GP consistently ranked worst**. The strong typing constraint (forcing boolean output) was the single most impactful design choice.

15.5.8 Relevance to NAT

The vectorial representation is directly applicable to NAT’s feature algebra: instead of applying transformations to scalar feature values, operate on fixed-length vectors (e.g., 100-tick windows) with vector-native operations (DOT, MEAN, STD). The strongly-typed approach suggests that constraining the Feature Factory Agent’s operation grammar (e.g., requiring that final outputs are bounded signals, not arbitrary reals) improves evolution quality. The overfitting mitigation (rotating evaluation segments) is a pattern NAT’s agent should adopt.

15.6 NNGPT: LLM as Self-Improving AutoML Engine

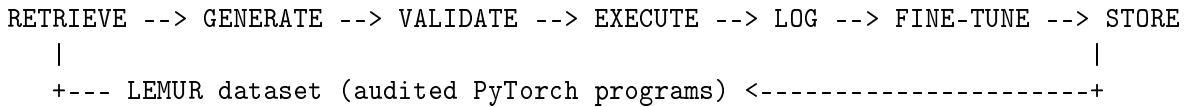
Paper: Kochnev, R., Khalid, W., Uzun, T.A. et al. (2025). NNGPT: Rethinking AutoML with Large Language Models. *arXiv:2511.20333*. November 2025.

15.6.1 Core Concept

NNGPT wraps an LLM (DeepSeek-R1, CodeLlama-7B) into a self-improving AutoML engine. Unlike search-based AutoML (Optuna, hundreds of trials) or NAS (DARTS, ENAS, compute-intensive), it generates **complete executable training specifications** — architecture code, preprocessing transforms, optimizer config, loss, metrics — from a single natural-language prompt, then **learns from execution outcomes** via LoRA fine-tuning and RL.

15.6.2 Closed-Loop Architecture

Seven-stage cycle:



Formally: generator G_θ maps prompt P and retrieval set R to executable configuration C ; validator V checks structural correctness; executor E produces metrics m ; predictor H_ϕ estimates final accuracy from early epochs. Every 50 completed runs, LoRA adapters are fine-tuned for 3 epochs (linear warmup + cosine decay).

15.6.3 Five Integrated Pipelines

#	Pipeline	Description
1	Zero-shot architecture synthesis (FSAP)	1–6 few-shot exemplars from LEMUR; MD5 deduplication. Produced $\sim 1,900$ unique architectures out of $\sim 5,000$ trained models. Mean accuracy 53.1% vs. 51.5% baseline; +11.6pp on CIFAR-100.
2	Hyperparameter optimization	LLM generates complete HP sets (lr, momentum, weight decay, batch size). 93% schema validity. RMSE 0.603 vs. Optuna’s 0.636 — one-shot generation eliminates 20,000+ Optuna trials.
3	Code-aware accuracy prediction	4-bit QLoRA model consumes metadata + code + epoch-1–2 accuracies. RMSE 0.1449, Pearson $r = 0.7766$ across 1,200+ runs. $\sim 50\%/75\%$ of predictions within 5%/10% of true accuracy.
4	NN-RAG	Retrieval-augmented patching: scans repositories for <code>nn.Module</code> subclasses, syntax-preserving extraction via LibCST, SHA-1 content addressing. 73% executability on 1,289 target blocks.
5	RL closed loop	Masked-architecture corpus where layer definitions are replaced by placeholders; model reconstructs them. GRPO-style policy-gradient updates to LoRA adapters. Parallel channel-mutation via <code>torch.fx</code> tracing.

15.6.4 Self-Improvement Mechanism

The key innovation is the **execution feedback loop**: every generated configuration is executed end-to-end, metrics are logged, and the LLM’s LoRA weights are updated on the accumulated (prompt, config, outcome) tuples. This means the model improves at generating high-performing configurations over time, without explicit search. The accuracy predictor (H_ϕ) enables **early stopping** of unpromising runs, reducing compute waste.

The RL component adds a reward signal mixing syntactic validity, runtime feasibility, and validation accuracy:

$$r = w_1 \cdot \mathbb{I}[\text{valid syntax}] + w_2 \cdot \mathbb{I}[\text{runs without error}] + w_3 \cdot \text{val_accuracy} \quad (86)$$

GRPO-style policy gradients update the LoRA adapters toward configurations that execute correctly and achieve high accuracy.

15.6.5 Key Results

Metric	NNGPT	Baseline
Architecture diversity	1,900 unique / 5,000 runs	N/A
HPO RMSE	0.603	Optuna: 0.636
Accuracy prediction (r)	0.7766	N/A
Schema validity	93%	N/A

15.6.6 Limitations

- Single-GPU (24GB) bottleneck limits context window and schema complexity
- Scope restricted to mid-scale vision (CIFAR, MNIST); ImageNet deferred
- No explicit novelty or hardware-efficiency objective
- RL underperforms on hard regimes (CIFAR-100: 0.016 vs. 0.208 baseline)
- Accuracy predictor correlation varies sharply across datasets

15.6.7 Relevance to NAT

NNGPT’s closed-loop pattern (generate → execute → log → fine-tune) maps directly to the Feature Factory Agent:

NNGPT Component	NAT Equivalent
LEMUR dataset	Parquet feature archive
Architecture generation	Feature operation DAG generation
Execution + metrics	IC/MI/ICIR evaluation
LoRA fine-tuning	Bias generator toward productive operations
Accuracy predictor	Early-stop unpromising features (ICIR < 0.2 after 1 window)
RL reward	Composite: IC significance + cost viability + novelty

The key transferable idea is **learning to generate better candidates from execution history**, not just filtering bad ones. This closes the gap between NAT’s current “search and filter” approach and a true self-improving system.

15.7 Synthesis: What NAT Should Adopt

From	Concept	How to Apply in NAT
QuantEvolve	Quality-diversity archive (MAP-Elites)	Bin features by regime, frequency, IC, turnover — maintain diversity, not just top performers
QuantEvolve	Multi-agent specialization	Data Agent (parquet analysis) → Research Agent (hypothesis) → Code Agent (feature DAG) → Eval Agent (IC/MI)
Vectorial GP	Vector-native operations	Operate on 100-tick windows with DOT, MEAN, STD as first-class GP operators
Vectorial GP	Strong typing	Constrain feature DAG output to bounded signals; boolean root for entry/exit
Vectorial GP	Rotating evaluation segments	Evaluate on rotating OOS windows to mitigate overfitting
NNGPT	Execution feedback fine-tuning	Track which operation types/params produce survivors; bias future generation
NNGPT	Early-stop predictor	Kill candidate features with $ICIR < 0.2$ after first evaluation window
NNGPT	RL reward shaping	Composite reward: IC + cost viability + novelty + temporal stability

16 Recommended Study Roadmap

Priority	Subject	Reference	NAT Relevance
1	Information Theory	Cover & Thomas, <i>Elements of Information Theory</i>	Feature selection, IT engine
2	KSG MI Estimation	Kraskov et al. (2004), PRE 69(6)	Core of alpha discovery
3	Kalman Filtering	Särkkä, <i>Bayesian Filtering and Smoothing</i> (free PDF)	3 algorithms use Kalman
4	FIR/IIR Filter Design	Oppenheim & Willsky, <i>Signals and Systems</i> Ch. 5–7	Feature Factory extension
5	Hawkes Processes	Bacry et al. (2015) review paper	Trade clustering model
6	Market Microstructure	Bouchaud et al., <i>Trades, Quotes, and Prices</i>	Kyle, Amihud, VPIN, propagator
7	Evolutionary Computation	Koza, <i>Genetic Programming</i>	Autonomous agent evolution
8	Sequential Analysis	Wald (1947), Tartakovsky <i>Sequential Analysis</i>	SPRT optimal entry
9	Hidden Markov Models	Rabiner (1989) tutorial	Regime detection extension
10	Transfer Entropy	Schreiber (2000), PRL 85(2)	Causal feature selection

Appendix: Feature Count Summary

Category	Count	Type
Raw	10	Base (always computed)
Imbalance	8	Base
Flow	12	Base
Volatility	9	Base
Entropy	24	Base
Context	12	Base
Trend	15	Base
Illiquidity	12	Base
Toxicity	10	Base
Derived	15	Base
Microstructure	5	Base
Resilience	3	Base
Hawkes	3	Base
Base subtotal	138	
Whale Flow	12	Optional
Liquidation Risk	13	Optional
Concentration	15	Optional
Regime	20	Optional
GMM Classification	8	Optional
Cross-Symbol	3	Optional
Optional subtotal	71	
Total features	209	
Algorithm-derived	56+	Streaming